
Integrative Programming and Technologies III

Test 2 (Design patterns, XML and data exchange, and Network programming)

Total: 35 marks

19 May 2023 Time: 1 hour

Questions 1-7

7 marks

1. Distributed applications architecture makes it easier to add new components. (T)
2. XML documents must be encoded for data exchange between different platforms. (T)
3. DTD translates how the data in an XML document should be displayed in a web page. (F)
4. The RMI software hides the low-level communication between the client and server from the programmer. (T)
5. In the web service architecture, it is objects that are published in the registry. (F)
6. In a client-server application, it is the client listens for requests from the server. (F)
7. RMI is based on the client-server architecture. (F)

Questions 8-9

4 marks

1. “Distributed application consist of independent processes”. Briefly explain what is an **independent process**. (2 marks)

Any correct answer, e.g., Independent process is a standalone, executable process. In a distributed environment, it performs a specific business process. ✓✓

2. Why is exception handling an important part of distributed applications? (2 marks)

Distributed applications communicate with each other over networks. There are several factors which may cause an application to terminate abruptly, e.g., connecting to a server which is down. Exception handling ensures applications do not terminate abruptly. ✓✓

Question 10

6 marks

The class below is an implementation of the **Singleton** pattern.

```
public class Printer {
    private static Printer p;
    private Printer() {

    }
    public static Printer getInstance() {
        if(p==null) {
            p=new Printer();
        }
        return p;
    }
}
```

1.1. Explain why the method `getInstance()` is declared static. (2 marks)

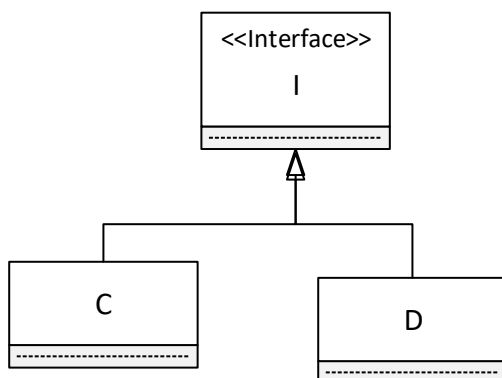
Answer: This is because it uses a static variable (p) ✓✓

1.2. The following code creates an instance of `Printer`. State whether the statement is **valid** or **invalid**. Explain your answer. (2 marks)

```
public class Tester {
    public static void main(String[] args) {
        Printer p1=new Printer();
        ....
    }
}
```

Answer: Invalid; ✓this is because the constructor is private and cannot be called from outside the class. ✓

2. The classes **C** and **D** implement the interface **I**. You want to create instances of the subclasses. Which design pattern provides guidelines. Explain your answer. (2 marks)



Answer: Factory pattern. This pattern provides guidelines for creating instances of subclasses. ✓✓

Question 11

10 marks

Below is part of an XML document (**contact.xml**). Each **Contact** has **id** (a unique identifier), **name**, **phone** number and **address** (which consists of **street** and **city**).

```
.....
<PhoneBook>
  <Contact id="1">
    <name>K Lo</name>
    <phone>046 222 2222</phone>
    <address>
      <street>11 Osmond Terrace</street>
      <city>Grahamstown</city>
    </address>
  </Contact>
  .....
</PhoneBook>
```

1. Complete the DTD document (**contacts.dtd**) below, which defines the structure of the data in **contacts.xml**. (6 marks)

```
<?xml version="1.0" encoding="UTF-8"?>
<!ELEMENT PhoneBook (Contact)+>
<!ELEMENT Contact (name,phone,address)>
<!_____ id _____> (2 marks)
<!ELEMENT name (#PCDATA)>
<!ELEMENT phone (#PCDATA)>
_____ (2 marks)
_____ (1 mark)
_____ (1 marks)
```

Answer:

```
<?xml version="1.0" encoding="UTF-8"?>
<!ELEMENT PhoneBook (Contact)+>
<!ELEMENT Contact (name,phone,address)>
<!ATTLIST Contact ✓id CDATA #REQUIRED> ✓
<!ELEMENT name (#PCDATA)>
<!ELEMENT phone (#PCDATA)>
<!ELEMENT address (street,city)> ✓✓
<!ELEMENT street (#PCDATA)> ✓
<!ELEMENT city (#PCDATA)> ✓
```

2. Complete the tag below that links the DTD to the XML document. (2 marks)

```
<!DOCTYPE _____ SYSTEM _____>
```

Answer:

```
<!DOCTYPE PhoneBook ✓ SYSTEM "contacts.dtd" ✓>
```

3. Below is part of the code to retrieve the **Contact** objects in the XML document. It creates an instance of **Contact** and calls **setId(String)** to set the value of the “**id**” attribute. Assume all variables have been declared. If **xmlreader** is an instance of

XMLStreamReader, complete the missing code to retrieve and set the value of “id”. (2 marks)

```
.....  
case XMLStreamConstants.START_ELEMENT:{  
    element=xmlreader.getLocalName();  
    if(element.equals("Contact")) {  
        Contact contact=new Contact();  
        contact.setId(_____);  
    }  
}
```

```
if(element.equals("Contact")) {  
    contact=new Contact();  
    contact.setId(xmlreader.getAttributeValue(0) ✓✓);  
}
```

Question 12

8 marks

Below is the class **ServerMark** (), a server-side program.

```

ServerMark
  ● response() : void
  ● status(double) : String

```

The method **status(double)** takes a **mark** (i.e., final mark) and returns the results (e.g., “Failed”, “Passed”, or “Distinction”). The method **response()** reads **studId** (student id, a string) and **mark** (final mark, double) from a socket, calls **status(double)** and writes back the **studId**, **mark** and **results** (e.g., Student ID=202020 Final mark=70.0 Status =Passed) to the socket.

Complete the missing parts of **response()** below.

```

public void response() throws Exception{
    ServerSocket server=new ServerSocket(1989);
    Socket socket=server.accept();

    _____
    _____
    _____
    _____

    server.close();
}

```

Answer:

```

public void response() throws Exception{
    ServerSocket server=new ServerSocket(1989);
    Socket socket=server.accept();
    //Set up streams; read id and mark
    InputStream is=socket.getInputStream();✓
    DataInputStream dis=new DataInputStream(is); ✓
    String id=dis.readUTF();✓
    double mark=dis.readDouble(); ✓
    //Call the methods to write id and
    OutputStream os=socket.getOutputStream();✓
    DataOutputStream dos=new DataOutputStream(os); ✓
    dos.writeUTF("\nStudent ID: "+id+"\tFinal mark "+ mark +"Status =" +status(mark)
    ✓✓);
    server.close();
}

```